

Reviewer Pack — Matroid Connectivity Determines Pseudorandom Generator Effic...

Entry f4e06590d9e7 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 09:19:37 UTC

Matroid Connectivity Determines Pseudorandom Generator Efficiency for Nisan-Wigderson Designs

Entry ID: f4e06590d9e7 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 09:19:37 UTC

1. Conjecture

Field A (mathematical branch): MATROID_THEORY **Field B** (complexity object): PSEUDORANDOMNESS

Statement:

Let M be a matroid of rank r over a ground set of size n , and let G be a Nisan-Wigderson pseudorandom generator with seed length s . The connectivity $\kappa(M)$ of M is inversely proportional to the seed length s required for G to fool circuits of depth $d = O(\log n)$. Specifically, for matroids with $\kappa(M) \geq \log n$, the seed length s is $\Omega(\log n)$, while for matroids with $\kappa(M) < \log n$, s is $\Omega(n/\kappa(M))$.

Rationale (proposer's reasoning):

Matroid connectivity (κ) measures the minimal number of elements whose removal disconnects the matroid. This structural property directly influences the expansion properties of the matroid's incidence graph, which in turn determines the pseudorandom generator's ability to fool circuits. High connectivity implies better expansion, requiring shorter seeds, while low connectivity necessitates longer seeds to maintain pseudorandomness.

Taxonomy category: NISAN_WIGDERSON_DESIGNS (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 2e73090cb07ae71c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	0.95	SAFE	HITS
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def extended_gcd(a, b):
    if a == 0:
        return b, 0, 1
    gcd, x1, y1 = extended_gcd(b % a, a)
    x = y1 - (b // a) * x1
    y = x1
    return gcd, x, y

def mod_inverse(a, m):
    gcd, x, _ = extended_gcd(a, m)
    if gcd != 1:
        raise ValueError("Inverse doesn't exist")
    else:
        return x % m

def matrix_multiply(A, B):
    n = len(A)
    C = [[0 for _ in range(n)] for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def gaussian_elimination(A, b):
    n = len(A)
    augmented_matrix = [A[i] + [b[i]] for i in range(n)]
    for i in range(n):
```

```

max_row = i
for j in range(i+1, n):
    if abs(augmented_matrix[j][i]) > abs(augmented_matrix[max_row][i]):
        max_row = j
augmented_matrix[i], augmented_matrix[max_row] = augmented_matrix[max_row], augmented_matrix[i]
pivot = augmented_matrix[i][i]
for j in range(i, n+1):
    augmented_matrix[i][j] /= pivot
for j in range(n):
    if j != i:
        factor = augmented_matrix[j][i]
        for k in range(i, n+1):
            augmented_matrix[j][k] -= factor * augmented_matrix[i][k]
x = [0] * n
for i in range(n-1, -1, -1):
    x[i] = augmented_matrix[i][-1]
    for j in range(i+1, n):
        x[i] -= augmented_matrix[i][j] * x[j]
return x

def matroid_connectivity(M):
    n = len(M)
    parent = list(range(n))
    rank = [0] * n
    def find_set(parent, u):
        if parent[u] != u:
            parent[u] = find_set(parent, parent[u])
        return parent[u]
    def union_sets(parent, rank, x, y):
        root_x = find_set(parent, x)
        root_y = find_set(parent, y)
        if rank[root_x] < rank[root_y]:
            parent[root_x] = root_y
        elif rank[root_x] > rank[root_y]:
            parent[root_y] = root_x
        else:
            parent[root_y] = root_x
            rank[root_x] += 1
    for e in M:
        u, v = e
        if find_set(parent, u) != find_set(parent, v):
            union_sets(parent, rank, u, v)
    components = [0] * n
    for i in range(n):
        components[find_set(parent, i)] += 1
    return min(components)

def simulate_nisan_wigderson_generator(s, d):
    # Placeholder function to simulate the generator
    # This is a stub and should be replaced with actual implementation
    return random.random() < s

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    M = [sorted(random.sample(range(n), 2)) for _ in range(n-1)]

```

```

κ_M = matroid_connectivity(M)
d = math.ceil(math.log2(n))

s1 = math.log2(n)
s2 = n / κ_M

pseudorandomness1 = simulate_nisan_wigderson_generator(s1, d)
pseudorandomness2 = simulate_nisan_wigderson_generator(s2, d)

metric_name = "pseudorandomness"
metric_value = abs(pseudorandomness1 - pseudorandomness2)
instances_tested = 2
conjecture_holds = False
counterexample = ""

if κ_M >= math.log2(n):
    if s1 <= s2:
        conjecture_holds = True
elif κ_M < math.log2(n):
    if s2 <= s1:
        conjecture_holds = True

return {
    "metric_name": metric_name,
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        counterexample = "pseudorandomness_difference"
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b61de3bf.py", line 143, in <module>
    trial_result = run_trial(seed)
                   ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b61de3bf.py", line 107, in run_trial
    k_M = matroid_connectivity(M)
         ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b61de3bf.py", line 91, in matroid_conne
    if find_set(parent, u) != find_set(parent, v):
       ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b61de3bf.py", line 76, in find_set
    if parent[u] != u:
       ~~~~~
```

IndexError: list index out of range

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with IndexError in matroid connectivity calculation, preventing data collection | next: Fix the index out-of-range error in the find_set function and rerun tests

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	118628
2	propose	ollama_remote	qwen3:8b	0	0	152883
3	preregistration	ollama_remote	qwen3:8b	0	0	27435
4	novelty	ollama_remote	qwen3:8b	0	0	24051
5	novelty	ollama_remote	qwen3:8b	0	0	16596
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24798
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7633
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13516
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15840
10	judge	ollama_remote	qwen3:8b	0	0	21827

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 423208 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in [research/audit/f4e06590d9e7.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f4e06590d9e7.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/f4e06590d9e7.tar.gz` (if generated)